

Automated Lending Platform

Architecture Recommendation

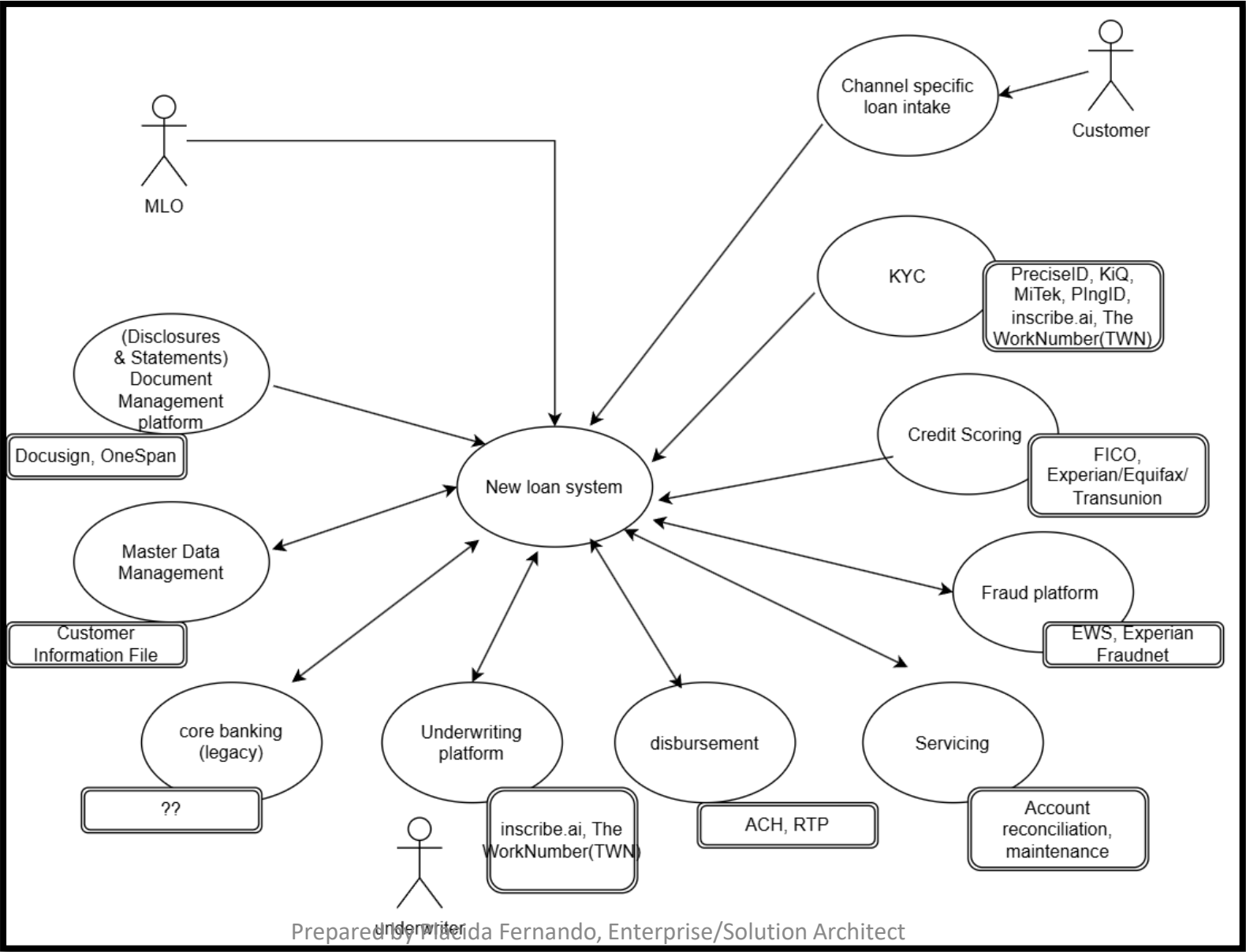
Problem Statement

Today, loan applications arrive at the bank as manual, paper-based submissions that staff must key in and shepherd through each step by hand. The bank wants to change that, automating the loan intake process end to end, along with the downstream processes it feeds: booking, servicing, and disbursement.

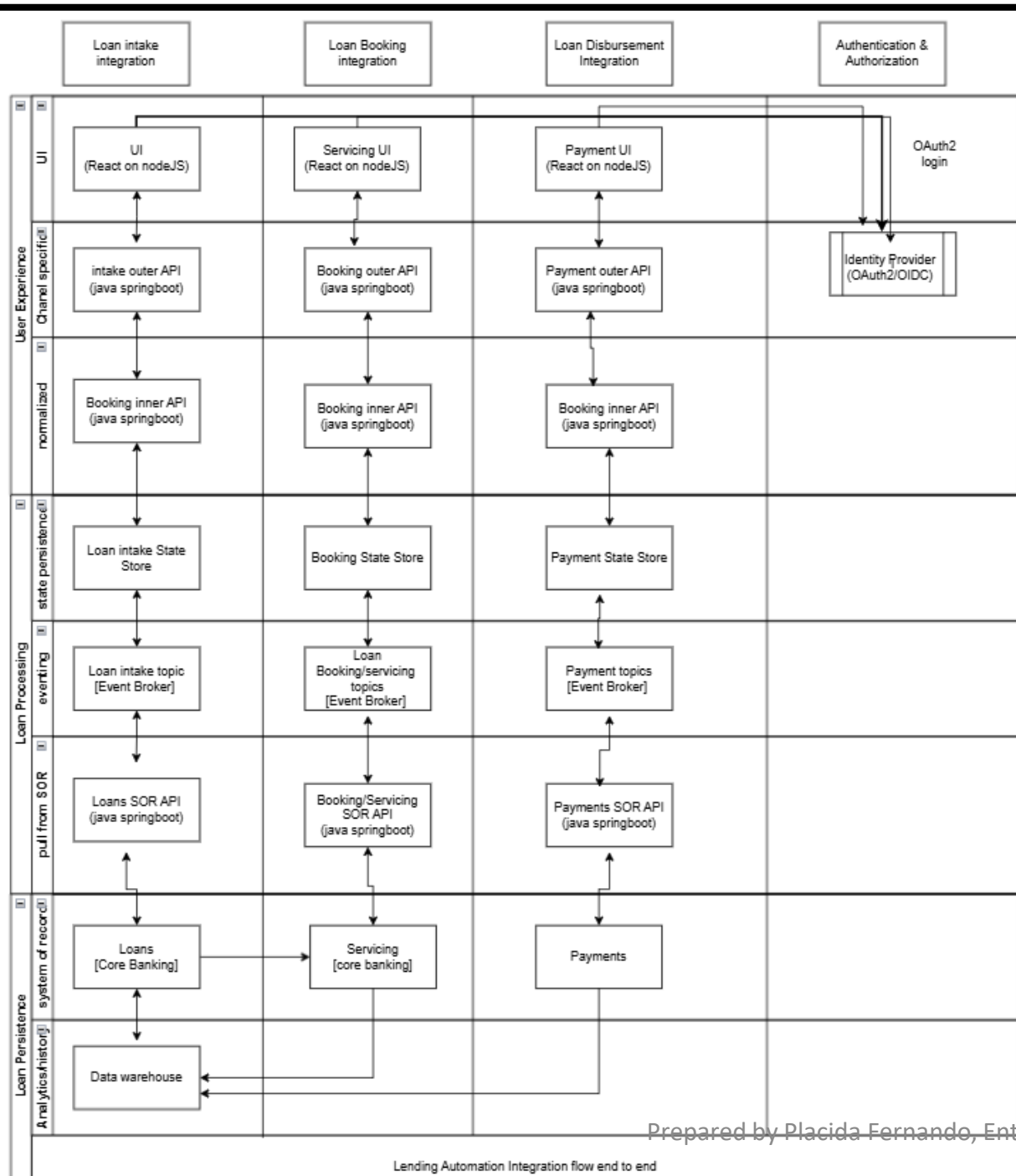
Architecture References

The solution is presented across three complementary views: a logical and business data architecture, an integration architecture that decouples processes along domain-driven design principles, and a container-based deployment view. The deployment view is intentionally infrastructure-agnostic, it does not name specific servers or hosting environments, as the containerized design runs equally well on premises or on any major cloud platform (AWS, Azure, or GCP).

Context Diagram – Level 0



Integration Diagram – Level 1

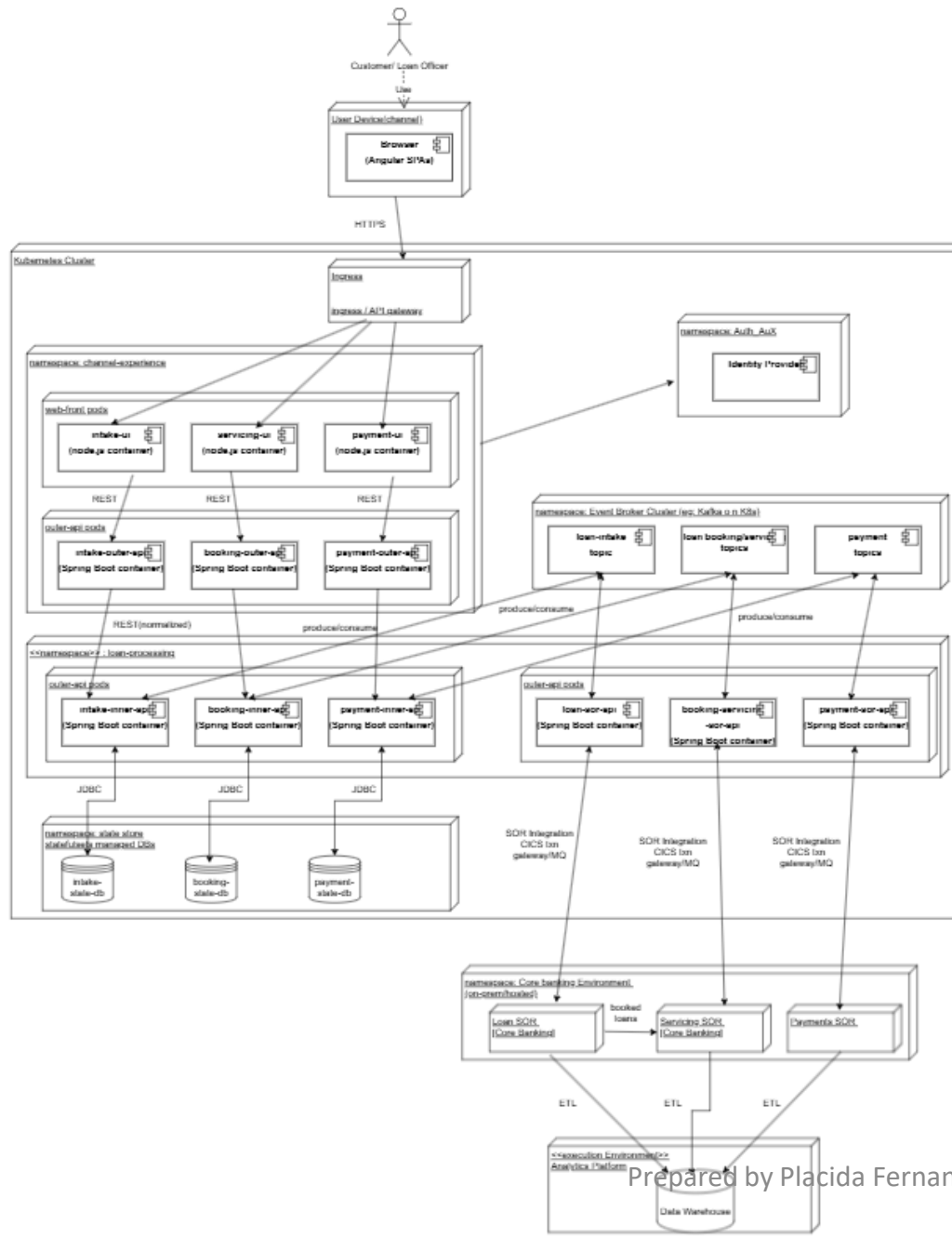


I designed as a layered n-tier architecture separating UI, channel-specific API, normalized API, state store, eventing, SOR API, system of record. Out of all the modules shown in my context diagram I depicted only loan intake, booking /servicing and loan disbursement (ACH or RTP rails)

Backend-for-Frontend + Anti-Corruption Layer is what my outer/inner API split is. The "channel specific" outer API is a BFF, an API shaped for one particular UI. The "normalized" inner API in DDD we call an anti-corruption layer): it translates between my clean domain model and the quirks of the core banking systems underneath, so the ugliness of the SOR doesn't leak upward which I believe as a good architecture.

Event-driven architecture with event-carried state transfer is my middle band. The flow of SOR API → topic → state store is a recognized pattern: instead of hitting core banking every time the UI needs data, changes are published as events and materialized into a local state store that serves reads fast. I built this as a form of CQRS-lite, reads come from the state store, writes go down through the APIs to the SOR

Level 2 - Deployment Diagram



I assumed the bank will use kubernetes even on an on-premis data center!!

I built multiple different kubernetes clusters for reusability purposes

Security

- The outer/inner API split creates natural enforcement points: the outer API is where you terminate Transport layer security, authenticate the user (OAuth2/OIDC tokens from the React/Angular SPA), and apply channel-specific throttling
- The inner API is where my enforce service-to-service auth (mutual transport layer security or client-credentials tokens), so nothing reaches the SOR APIs without passing two checkpoints.
- The fact that UIs never touch core banking directly, everything funnels through the API layers, is defense in depth by construction.

Scalability

- Every service layer (UIs, outer APIs, inner APIs, SOR APIs) is stateless and containerized on Kubernetes, so each scales horizontally and independently, you can run 10 intake-outer-api pods during a promotion campaign without touching booking or payments.
- State lives in dedicated stores and topics, not in the services, that's what makes the horizontal scaling safe (any pod can handle any request).
- The event broker (Kafka-style) decouples producers from consumers, so a spike in loan applications queues up rather than overwhelming the SOR APIs; consumers drain at their own pace.
- The state stores absorb read traffic that would otherwise hit core banking, the layer that scales worst. Core banking is protected behind the SOR APIs, which can throttle and batch.
- The vertical-slice design means each domain (intake, booking, disbursement) scales to its own demand profile, intake is bursty, payments are steady.

Availability

- No single point of failure in the middle tiers: multiple pod replicas per service behind Kubernetes services, with self-healing (failed pods are restarted/rescheduled automatically).
- The event broker provides durability, if the payments SOR API is down, payment events persist in the topic and process when it recovers; nothing is lost, the flow degrades to "delayed" instead of "failed."
- The state stores allow the UI to keep serving reads (application status, loan details) even when core banking is offline, core banking outages become invisible for read traffic.
- Async event flows mean a failure in one column doesn't cascade to the others — intake keeps taking applications even if disbursement is down.
- Caveats to acknowledge: the core banking systems and the event broker itself are the availability-critical components - the broker needs multi-broker replication, and the state stores need replicas/failover, or they become the new single points of failure.

Interoperability

- The normalized inner API layer is the key: it exposes one canonical loan model regardless of what sits underneath, so a core banking system could be swapped (or a second one added, e.g. after a merger) with changes confined to the SOR API layer.
- Standard, ubiquitous protocols everywhere, REST/JSON for synchronous calls, OIDC/OAuth2 JWT for identity, event messages on the broker, so any new channel or partner system can integrate without custom adapters.
- The outer API (BFF) layer means new channels, mobile app, broker portal, third-party aggregator, plug in by adding a new outer API against the same inner APIs, without touching the domain logic.
- Events on the broker are an integration surface in themselves: a new consumer (fraud detection, notifications, a partner feed) subscribes to existing topics with zero changes to producers.
- The ETL path into the warehouse gives analytics and regulatory tools a standard SQL surface, keeping BI tooling decoupled from operational systems.